The Scheme 311 Compiler
An Exercise in Denotational Semantics

William Clinger
Indiana University

Introduction.

Many authors have offered much good advice on structuring compilers and proving them correct [3, 12, 13, 27, 28]. The correctness proof described here demonstrates that their advice can be applied to a useful compiler for a real programming language. This paper describes and proves the correctness of a simple compiler algorithm for Scheme, a statically scoped dialect of Lisp. The algorithm has been used as the basis for an interactive compiler generating interpreted byte code in an implementation comparable to those available for Pascal, Smalltalk, and Basic [1, 7, 9].

Most of the correctness proof consists of trivial calculations. This is possible because the meanings of target code instructions are expressed in the same language used to express source meanings, and that language can be wielded as a calculus. The proof is complicated by what amounts to a static type distinction needed to compile primitive operators in line. Previous compiler correctness proofs using static type information (eg [17]) have assumed that separate presentations of static and dynamic semantics are available a priori, but the proof in this paper works from a single standard semantics.

The proof is similar in spirit to that of [29], though the algorithm was designed and a compiler built before any thought was given to a formal correctness proof. The algorithm is superior to that in [29] in that it directly produces linear and properly tail-recursive object code of reasonable quality.

The Language.

Scheme is a statically scoped dialect of Lisp. Parameters are passed by value. Procedures, including escape procedures in full Scheme, are first class objects that may be returned as results, stored in variables, and so on.

Dynamically scoped "fluid" variables are available in the full language but are omitted from this paper because they lengthen the presentation without adding new insights.

As seen by the compiler, the core syntax of Scheme comprises constants, identifier references, assignments, lambda expressions, conditional expressions, applications, and sequenced expressions. These language constructs compose well enough that most other syntactic constructs can be expressed as compile time macros, and most operators can be considered to be predeclared procedures. Indeed even the operator that creates escape procedures is considered to be a procedure rather than a syntactic construct.

The standard semantics of Scheme was described in [15], and a simplified and updated version of that semantics is used in this paper [Figure 1]. In two cases, however, the semantics used here has been fudged a bit to reflect the compiler's quirks. Firstly, Scheme does not really specify order of evaluation, but the Scheme 311 compiler, for historical reasons, evaluates arguments right to left. Secondly, the compiler translates calls to primitive operators in line for the sake of efficiency. In most languages this is easy because the primitive operators are constants. In Scheme, however, + is merely an identifier denoting a location that happens to hold the primitive procedure for addition. Rather than introduce a horde of reserved identifiers, Scheme 311 makes + denote the primitive procedure directly in the initial environment. Programmers can still bind + locally and give it whatever value they wish.

The compiler algorithm is indifferent to the primitive procedures and data types, so they are not described in Figure 1. The data types may be specified by fixing a domain equation for the domain G of "other expressed values". For example a dialect of Scheme with generic numbers, symbols, mutable conses and vectors, and immutable strings could use the domain equation

$$G = R + Q + L \times L + L^* + H^*$$

where R, Q, and H are independently specified domains of numbers, symbols, and characters. The primitive and predeclared procedures may be specified by describing the initial environment ρ^{init} and the initial store σ^{init}.

It is significant that Scheme officially has no **eval** operator. The compiler correctness theorem of this paper depends on the homomorphic nature of the semantics as codified by the *principle of compositionality:* the semantics of a program fragment is an independently definable function of the semantics of its syntactic constituents. **eval** violates this principle because the semantics of an expression involving **eval** depends on the "semantics" of data structures constructed at run time.

Certain technical matters should be considered by anyone wishing to add **eval** as an extension. An **eval** that evaluates in the current lexical environment cannot be added to Scheme as a predeclared procedure, since procedures do not have access to such environments. A flavor of **eval** that evaluates in a fixed environment or that takes a representation of an environment as an argument could be a predeclared procedure, but in order to preserve the principle of compositionality its semantics would have to be definable without reference to the semantic function $\mathcal{E}$. One approach would be to define it not in terms of $\mathcal{E}$ but in terms

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

© 1984 ACM 0-89791-142-3/84/008/0356 $00.75

of a "copy" $\mathcal{E}'$. (Think of $\mathcal{E}$ as the compiler's semantics and $\mathcal{E}'$ as an interpreter's semantics.) $\mathcal{E}'$ need not be compositional, so the semantics of **eval** in $\mathcal{E}'$ may without difficulty refer circularly to $\mathcal{E}'$. Alternatively one may imagine an infinite sequence of functions $\mathcal{E}, \mathcal{E}', \mathcal{E}'', \ldots$ as in [20].

The Target Machine.

The Scheme 311 compiler algorithm as described here produces code for a hypothetical and rather abstract Scheme machine. This machine's architecture is guided but not determined by the semantics of Scheme.

If correctness were the only issue, the machine's design would be a trivial matter. Its instructions could be taken to correspond to operations of the multi-sorted algebra generated by the semantics [14]. The difficulty lies in designing a machine that can easily be proved correct and will also run acceptably fast.

An implementation's correctness depends on the harmony and correctness of many parts operating at different levels of abstraction. Its designer must choose representations for each level that are sufficiently concrete to express what needs to be expressed at that level and at the same time sufficiently abstract to be tractable and to preserve design flexibility at lower levels.

Figure 1. Semantics of Scheme (less fluids)

Syntactic domains

$I \in$	**Ide**	identifiers
$K \in$	**Con**	constants
$E \in$	**Exp**	expressions

Productions

$$E ::= K \mid I \mid \textbf{(set!}\ I\ E\textbf{)} \mid \textbf{(lambda (}I^*\textbf{)}\ E\textbf{)} \mid \textbf{(if}\ E_0\ E_1\ E_2\textbf{)} \mid (E_0\ E^*) \mid \textbf{(begin}\ E_0\ E_1\textbf{)}$$

Value domains

$\alpha \in$ **L**	locations
$\nu \in$ **N**	natural numbers
T = $\{true\} + \{false\}$	booleans
$\phi \in$ **F** = $\mathbf{E}^* \to \mathbf{K} \to \mathbf{C}$	procedure values
G	other expressed values
$\beta \in$ **V** = **E**	stored values
$\epsilon \in$ **E** = $\{nil\} + \mathbf{F} + \mathbf{G}$	expressed values
$\delta \in$ **D** = **F** + **L**	denoted values
$\sigma \in$ **S** = **(L** $\to$ **(V x T))**	stores
$\rho \in$ **U** = **Ide** $\to$ **D**	environments
A	answers
$\theta \in$ **C** = **S** $\to$ **A**	command continuations
$\kappa \in$ **K** = **E** $\to$ **C**	expression continuations

Semantic functions

$\mathcal{K}$: **Con** $\to$ **E**
$\mathcal{E}$: **Exp** $\to$ **U** $\to$ **K** $\to$ **C**
$\mathcal{E}^*$: **Exp** $\to$ **U** $\to$ $(\mathbf{E}^* \to \mathbf{C}) \to$ **C**

$\mathcal{K}$: **Con** $\to$ **E**

[definition deliberately omitted]

Figure 1 is continued on the next page

Figure 1 continued.

$\mathcal{E}$: Exp → U → K → C

$\mathcal{E}[\![\mathrm{K}]\!] = \lambda\rho\kappa\,.\,\kappa(\mathcal{K}[\![\mathrm{K}]\!])$

$\mathcal{E}[\![\mathrm{I}]\!] = \lambda\rho\kappa\,.\,\rho[\![\mathrm{I}]\!] \in \mathrm{F} \to \kappa(\rho[\![\mathrm{I}]\!] \mid \mathrm{F}\ \mathrm{in}\ \mathrm{E}),\ \mathit{hold}(\rho[\![\mathrm{I}]\!] \mid \mathrm{L})\kappa$

$\mathcal{E}[\![(\mathtt{set!}\ \ \mathrm{I}\ \mathrm{E})]\!] =$
$\quad \lambda\rho\kappa\,.\,\mathcal{E}[\![\mathrm{E}]\!]\rho(\lambda\epsilon\,.\,\rho[\![\mathrm{I}]\!] \in \mathrm{F} \to \mathit{wrong},\ \mathit{assign}(\rho[\![\mathrm{I}]\!] \mid \mathrm{L})\epsilon(\kappa\epsilon))$

$\mathcal{E}[\![(\mathtt{lambda}\ (\mathrm{I}^*)\ \mathrm{E})]\!] =$
$\quad \lambda\rho\kappa\,.\,\kappa((\lambda\epsilon^*\kappa'\,.\,\#\epsilon^* = \#\mathrm{I}^* \to \mathit{tievals}(\lambda\delta^*\,.\,\mathcal{E}[\![\mathrm{E}]\!](\rho[\delta^*/\mathrm{I}^*])\kappa')\epsilon^*,\ \mathit{wrong})\ \mathrm{in}\ \mathrm{E})$

$\mathcal{E}[\![(\mathtt{if}\ \mathrm{E}_0\ \mathrm{E}_1\ \mathrm{E}_2)]\!] = \lambda\rho\kappa\,.\,\mathcal{E}[\![\mathrm{E}_0]\!]\rho(\lambda\epsilon\,.\,\epsilon = \mathit{nil} \to \mathcal{E}[\![\mathrm{E}_2]\!]\rho\kappa,\ \mathcal{E}[\![\mathrm{E}_1]\!]\rho\kappa)$

$\mathcal{E}[\![(\mathrm{E}_0\ \mathrm{E}^*)]\!] = \lambda\rho\kappa\,.\,\mathcal{E}^*[\![\mathrm{E}^*]\!]\rho(\lambda\epsilon^*\,.\,\mathcal{E}[\![\mathrm{E}_0]\!]\rho(\lambda\epsilon\,.\,\mathit{apply}\,\epsilon\,\epsilon^*\kappa))$

$\mathcal{E}[\![(\mathtt{begin}\ \mathrm{E}_0\ \mathrm{E}_1)]\!] = \lambda\rho\kappa\,.\,\mathcal{E}[\![\mathrm{E}_0]\!]\rho(\lambda\epsilon\,.\,\mathcal{E}[\![\mathrm{E}_1]\!]\rho\kappa)$

$\mathcal{E}^*$: Exp → U → (E* → C) → C

$\mathcal{E}^*[\![\ ]\!] = \lambda\rho\psi\,.\,\psi\langle\rangle$

$\mathcal{E}^*[\![\mathrm{E}^*\ \mathrm{E}]\!] = \lambda\rho\psi\,.\,\mathcal{E}[\![\mathrm{E}]\!]\rho(\lambda\epsilon\,.\,\mathcal{E}^*[\![\mathrm{E}^*]\!]\rho(\lambda\epsilon^*\,.\,\psi(\epsilon^* \S \langle\epsilon\rangle)))$

Auxiliary functions

wrong : S → A — [definition deliberately omitted]

new : S → (L + {*error*}) — [definition deliberately omitted]

hold : L → K → C

$\mathit{hold} = \lambda\alpha\kappa\,.\,\lambda\sigma\,.\,\kappa(\sigma\,\alpha \downarrow 1 \mid \mathrm{E})\sigma$

assign : L → E → C → C

$\mathit{assign} = \lambda\alpha\epsilon\theta\,.\,\lambda\sigma\,.\,\theta(\mathit{update}\,\alpha(\epsilon\ \mathrm{in}\ \mathrm{V})\sigma)$

update : L → V → S → S

$\mathit{update} = \lambda\alpha\beta\sigma\,.\,\lambda\alpha'\,.\,\alpha' = \alpha \to \langle\beta, \mathit{true}\rangle, \sigma\,\alpha'$

$\mathit{tievals}$: (D* → C) → E* → C

$\mathit{tievals} =$
$\quad \lambda\psi\epsilon^*\sigma\,.\,\#\epsilon^* = 0 \to \psi\langle\rangle\sigma,$
$\qquad \mathit{new}\,\sigma \in \mathrm{L} \to \mathit{tievals}(\lambda\delta^*\,.\,\psi(\langle\mathit{new}\,\sigma \mid \mathrm{L}\ \mathrm{in}\ \mathrm{D}\rangle \S \delta^*))$
$\qquad\qquad (\epsilon^* \dagger 1)$
$\qquad\qquad (\mathit{update}(\mathit{new}\,\sigma \mid \mathrm{L})(\epsilon^* \downarrow 1\ \mathrm{in}\ \mathrm{V})\sigma),$
$\qquad\quad \mathit{wrong}\,\sigma$

apply : E → E* → K → C

$\mathit{apply} = \lambda\epsilon\epsilon^*\kappa\,.\,\epsilon \in \mathrm{F} \to (\epsilon \mid \mathrm{F})\epsilon^*\kappa, \mathit{wrong}$

For programming languages the highest level of abstraction speaks of environments, continuations, stores, expressed values, and so on. The next highest level of abstraction begins to speak of implementation concepts. For example compile-time and run-time environments may be introduced, and environments may be represented as compositions of compile-time and run-time environments [Figure 2]. At this level of abstraction there is no need to commit to any particular representation of these new objects, so they should be treated as functions rather than as data structures.

For Scheme 311 this level introduces code segments as well. Code segments are taken to be values in the domain

$$\mathbf{P} = \mathbf{E} \to \mathbf{E}^* \to \mathbf{U}_R \to \mathbf{K} \to \mathbf{C}$$

where $\mathbf{U}_R$ is the domain of run-time environments [Figure 2]. The domain P determines the register architecture of the Scheme machine, since it is interpreted as the domain of maps from register contents to command continuations. Thus the Scheme machine has an accumulator capable of holding an expressed value, a stack of expressed values, a register to hold the run-time environment, and a register to hold the expression continuation. This was an arbitrary design choice. Other register architectures could have been used instead.

Continuations can now be represented in terms of code segments, sequences of expressed values, run-time environments, and continuations. The general form of a continuation in this new representation is

$$\lambda\epsilon\,.\ \pi\epsilon\epsilon^*\rho_R\kappa$$

where π is a code segment. The representation chosen for continuations reflects the fact that during a computation the effective continuation may actually be made up of the contents of all registers, and only the most long-term portion is held in the so-called continuation register or control stack.

The instructions of the Scheme machine are specified by detailing a continuation semantics for each instruction [Figure 4]. The general form of an instruction is a mapping from zero or more parameters to a mapping from register contents to command continuations. This style of semantic description is superior to a register transfer language description in two respects. The first is that proofs in continuation semantics tend to be more local, at a higher level, and hence more tractable, than proofs in a state transition semantics, and the second is that the exact effects of errors and exceptions are more easily accomodated within the framework of continuation semantics. At the same time a register transfer language description can easily be obtained from the continuation semantics should such a description be needed by a lower level of abstraction.

To compile tail recursion correctly it is important to distinguish among procedure invocation, continuation invocation, and continuation creation. The Scheme machine instructions corresponding to these notions are *invoke*, *restore*, and *save*.

The *fetch* and *lexical* instructions are identical except for the types of their parameters, and likewise the *set* and *setlex* instructions. The difference in parameter types may become important when these instructions are later encoded as bit strings.

Figure 2. Compile Time and Run Time Environments

$$\rho \in \mathbf{U} = \mathbf{Ide} \to \mathbf{D}$$
$$\rho_C \in \mathbf{U}_C = \mathbf{Ide} \to \mathbf{D}_C$$
$$\rho_R \in \mathbf{U}_R = \mathbf{D}_C \to \mathbf{D}$$

$$rep_C\colon \mathbf{U} \to \mathbf{U}_C$$

$$rep_R\colon \mathbf{U} \to \mathbf{U}_R$$

$$extends_C\colon \mathbf{U}_C \to \mathbf{Ide}^* \to \mathbf{U}_C$$

$$extends_R\colon \mathbf{U}_R \to \mathbf{D}^* \to \mathbf{U}_R$$

Axioms.

$$(rep_R\,\rho) \circ (rep_C\,\rho) = \rho$$
$$(extends_R\,\rho_R\,\delta^*) \circ (extends_C\,\rho_C\,\mathrm{I}^*) = (\rho_R \circ \rho_C)[\delta^*/\mathrm{I}^*]$$

Figure 3. Compile Time Denotations for Scheme

$$\delta_C \in \mathbf{D}_C = \mathbf{F} + \mathbf{Ide} + (\mathbf{N} \times \mathbf{N})$$

The reasonable run-time environment condition on ρ_R:

1. $(\delta_C \in \mathbf{F}) = (\rho_R\,\delta_C \in \mathbf{F})$
2. If $\delta_C \in \mathbf{F}$ then $\rho_R\,\delta_C = (\delta_C \mid \mathbf{F}\ \mathbf{in}\ \mathbf{D})$.

The Compiler.

The Scheme 311 compiler algorithm introduces an embryonic notion of static type by separating compile-time denoted values into three subdomains: primitive procedures, global variables represented as identifiers for the sake of the linker, and local variables represented as a block number and offset. Three inference rules needed in the correctness proof require that environment components behave reasonably on these subdomains; for example compile-time environments must be strict and run-time environments must be identities on the subdomain of primitive procedures [Figure 3]. The easiest way to enforce these conditions is to require that the operations on environment components preserve them.

These conditions are necessary because the static information has been extracted from a standard semantics that has no notion of compile time or run time. Similar conditions would be needed to prove the correctness of a compiler algorithm for a statically typed language. So far, however, it seems that most work with such languages assumes that the separation into static and dynamic semantics has already been done correctly.

The compiler algorithm [Figure 5] is defined in terms of maps

t:	$\mathbf{Exp} \to \mathbf{U}_C \to \mathbf{P} \to \mathbf{P}$	expression compiler, tidy
e:	$\mathbf{Exp} \to \mathbf{U}_C \to \mathbf{P} \to \mathbf{P}$	expression compiler
e^*:	$\mathbf{Exp}^* \to \mathbf{U}_C \to \mathbf{P} \to \mathbf{P}$	argument list compiler

Intuitively e is a compiling function that takes an expression E, a compile-time environment ρ_C, and a code segment π to follow the code for E, and produces a code segment consisting of the code for E followed by π. The reason π is given as an argument to e is that in some cases π will represent dead code and can be ignored by e, though in fact this happens only when π is a single *restore* instruction or when the programmer attempts to assign to an identifier bound directly to a procedure.

The Correctness Proof.

The code generated for a top level expression E denotes

$$e[\![\mathrm{E}]\!]\rho_C^{init} restore$$

where ρ_C^{init} is the compile-time component of the initial environment. Part B of the following theorem asserts that this code, when applied to a random value ϵ in the accumulator, an empty stack, and a reasonable run-time environment, will yield the standard meaning of the expression in the environment formed by composing the run-time and compile-time environments. This establishes the algorithm's correctness.

Theorem. If ρ_C is strict, ρ_R is a reasonable run-time environment, and the operations on compile-time and run-time environments preserve strictness and reasonableness, respectively, then

A. Tidiness.

$$t[\![\mathrm{E}]\!]\rho_C\pi\epsilon\epsilon^*\rho_R\kappa = \mathcal{E}[\![\mathrm{E}]\!](\rho_R \circ \rho_C)(\lambda\epsilon\ .\ \pi\epsilon\epsilon^*\rho_R\kappa)$$

B. Correctness.

$$e[\![\mathrm{E}]\!]\rho_C restore\ \epsilon\,(\,)\rho_R\kappa = \mathcal{E}[\![\mathrm{E}]\!](\rho_R \circ \rho_C)\kappa$$

C. Evlis. If $\forall \epsilon, \epsilon'\ \pi\epsilon = \pi\epsilon'$ then

$$e^*[\![\mathrm{E}^*]\!]\rho_C\pi\epsilon\epsilon^*\rho_R\kappa = \mathcal{E}^*[\![\mathrm{E}^*]\!](\rho_R \circ \rho_C)(\lambda\epsilon_1^*\ .\ \pi\epsilon(\epsilon_1^* \S \epsilon^*)\rho_R\kappa)$$

Sketch of proof: The proof proceeds by induction on E and E*. The most difficult part of the proof makes use of the following two lemmas to show part A for applications. Both lemmas are proved by induction on #E*.

Lemma 1.

$$\mathcal{E}^*[\![\mathrm{E}^*]\!]\rho(\lambda\epsilon^*\ .\ f(takefirst(\#\mathrm{E}^*)(\epsilon^* \S x))((\epsilon^* \S y) \dagger (\#\mathrm{E}^*))) = \mathcal{E}^*[\![\mathrm{E}^*]\!]\rho(\lambda\epsilon^*\ .\ f\epsilon^* y)$$

Figure 4. Scheme Machine Instructions

$$save = \lambda\pi_1\pi_2\ .\ \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ \pi_1\epsilon\,(\,)\rho_R(\lambda\epsilon\ .\ \pi_2\epsilon\epsilon^*\rho_R\kappa)$$

$$restore = \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ \kappa\epsilon$$

$$const = \lambda\epsilon_0\pi\ .\ \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ \pi\epsilon_0\epsilon^*\rho_R\kappa$$

$$fetch = \lambda\mathrm{I}\pi\ .\ \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ hold(\rho_R(\mathrm{I\ in\ D}_C) \mid \mathrm{L})(\lambda\epsilon\ .\ \pi\epsilon\epsilon^*\rho_R\kappa)$$

$$lexical = \lambda x\pi\ .\ \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ hold(\rho_R(x\ \mathrm{in\ D}_C) \mid \mathrm{L})(\lambda\epsilon\ .\ \pi\epsilon\epsilon^*\rho_R\kappa)$$

$$set = \lambda\mathrm{I}\pi\ .\ \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ assign(\rho_R(\mathrm{I\ in\ D}_C) \mid \mathrm{L})\epsilon(\pi\epsilon\epsilon^*\rho_R\kappa)$$

$$setlex = \lambda x\pi\ .\ \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ assign(\rho_R(x\ \mathrm{in\ D}_C) \mid \mathrm{L})\epsilon(\pi\epsilon\epsilon^*\rho_R\kappa)$$

$$lambda = \lambda\nu\pi_1\pi_2\ .\ \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ \pi_2((\lambda\epsilon^*\kappa'\ .\ \#\epsilon^* = \nu \to tievals(\lambda\delta^*\ .\ \pi_1(nil\ \mathrm{in\ E})(\,)(extends_R\,\rho_R\,\delta^*)\kappa')\epsilon^*, wrong)\ \mathrm{in\ E})\epsilon^*\rho_R\kappa$$

$$if = \lambda\pi_1\pi_2\ .\ \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ \epsilon = nil \to \pi_2\epsilon\epsilon^*\rho_R\kappa, \pi_1\epsilon\epsilon^*\rho_R\kappa$$

$$invoke = \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ apply\,\epsilon\,\epsilon^*\kappa$$

$$push = \lambda\pi\ .\ \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ \pi\epsilon((\epsilon) \S \epsilon^*)\rho_R\kappa$$

$$op = \lambda\nu\phi\pi\ .\ \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ apply(\phi\ \mathrm{in\ E})((\epsilon) \S (takefirst\,\nu\epsilon^*))(\lambda\epsilon\ .\ \pi\epsilon(\epsilon^* \dagger \nu)\rho_R\kappa)$$

$$illegal = \lambda\epsilon\epsilon^*\rho_R\kappa\ .\ wrong$$

Lemma 2.

$\mathcal{E}^*[\![\mathrm{E}\quad \mathrm{E}^*]\!]\rho\psi = \mathcal{E}^*[\![\mathrm{E}^*]\!]\rho(\lambda\epsilon^* .\ \mathcal{E}[\![\mathrm{E}]\!]\rho(\lambda\epsilon .\ \psi((\epsilon)\,\S\,\epsilon^*)))$

With the help of these two lemmas, part A for applications is not too much more difficult than the second most difficult part of the proof, which shows part A for **set!** expressions. That part is shown here in order to illustrate the role played by the conditions on environment components.

The right hand side of part A for **set!** expressions is

$\mathcal{E}[\![(\mathtt{set!}\quad \mathrm{I}\ \mathrm{E})]\!](\rho_R \circ \rho_C)(\lambda\epsilon .\ \pi\epsilon\epsilon^*\rho_R\kappa)$

$= \mathcal{E}[\![\mathrm{E}]\!](\rho_R \circ \rho_C)(\lambda\epsilon .\ (\rho_R \circ \rho_C)\mathrm{I} \in \mathrm{F} \to \mathit{wrong},$
$\quad \mathit{assign}((\rho_R \circ \rho_C)\mathrm{I} \mid \mathrm{L})\epsilon(\pi\epsilon\epsilon^*\rho_R\kappa))$

The left hand side is

$t[\![(\mathtt{set!}\quad \mathrm{I}\ \mathrm{E})]\!]\rho_C\pi\epsilon\epsilon^*\rho_R\kappa$

$= t[\![\mathrm{E}]\!]\rho_C(\rho_C\mathrm{I} \in \mathrm{F} \to \mathit{illegal},$
$\quad \rho_C\mathrm{I} \in \mathtt{Ide} \to \mathit{set}(\rho_C\mathrm{I} \mid \mathtt{Ide})\pi,$
$\quad \mathit{setlex}(\rho_C\mathrm{I} \mid \mathtt{N\ x\ N})\pi)\epsilon\epsilon^*\rho_R\kappa$

$= \mathcal{E}[\![\mathrm{E}]\!](\rho_R \circ \rho_C)(\lambda\epsilon .\ (\rho_C\mathrm{I} \in \mathrm{F} \to \mathit{illegal},$
$\quad \rho_C\mathrm{I} \in \mathtt{Ide} \to \mathit{set}(\rho_C\mathrm{I} \mid \mathtt{Ide})\pi,$
$\quad \mathit{setlex}(\rho_C\mathrm{I} \mid \mathtt{N\ x\ N})\pi)\epsilon\epsilon^*\rho_R\kappa)$
(by induction hypothesis)

$= \mathcal{E}[\![\mathrm{E}]\!](\rho_R \circ \rho_C)(\lambda\epsilon .\ \rho_C\mathrm{I} \in \mathrm{F} \to \mathit{wrong},$
$\quad \rho_C\mathrm{I} \in \mathtt{Ide} \to \mathit{assign}(\rho_R(\rho_C\mathrm{I} \mid \mathtt{Ide}\ \mathrm{in}\ \mathrm{D}_C) \mid \mathrm{L})\epsilon(\pi\epsilon\epsilon^*\rho_R\kappa),$
$\quad \mathit{assign}(\rho_R(\rho_C\mathrm{I} \mid \mathtt{N\ x\ N}\ \mathrm{in}\ \mathrm{D}_C) \mid \mathrm{L})\epsilon(\pi\epsilon\epsilon^*\rho_R\kappa))$

$= \mathcal{E}[\![\mathrm{E}]\!](\rho_R \circ \rho_C)(\lambda\epsilon .\ \rho_C\mathrm{I} \in \mathrm{F} \to \mathit{wrong},$
$\quad \mathit{assign}((\rho_R \circ \rho_C)\mathrm{I} \mid \mathrm{L})\epsilon(\pi\epsilon\epsilon^*\rho_R\kappa))$
(by strictness of ρ_C)

$= \mathcal{E}[\![\mathrm{E}]\!](\rho_R \circ \rho_C)(\lambda\epsilon .\ (\rho_R \circ \rho_C)\mathrm{I} \in \mathrm{F} \to \mathit{wrong},$
$\quad \mathit{assign}((\rho_R \circ \rho_C)\mathrm{I} \mid \mathrm{L})\epsilon(\pi\epsilon\epsilon^*\rho_R\kappa))$
(by reasonableness of ρ_R)

The remainder of the proof consists of similar calculations.

The peculiar hypothesis of part C is needed because argument evaluation destroys the contents of the accumulator. It is easy to verify that the hypothesis is satisfied whenever part C is used in the induction. This device can be used to ignore other harmless side effects such as to condition code settings on many hardware architectures. ∎

Tail Recursion.

The map e is used only in tail-recursive positions where registers need not be preserved. In non-tail-recursive positions a tidy version of e called t is used, which in effect wraps a save/restore sequence around code that would otherwise destroy registers. Actually the caller saves registers and a return address before evaluating untidy arguments. The callee restores registers when it returns. This calling discipline is based on [22], and is similar to calling conventions used in most other implementations of Scheme.

Thus the body of a tail-recursive procedure such as

```
(define add
  (lambda (x y)
    (if (0? x)
        y
        (add (1- x) (1+ y)))))
```

compiles as something like

```
        lexical  0,0          ;x
        0?
        branchf  L1           ;branch if nil
        lexical  0,1          ;y
        restore
L1:     lexical  0,1          ;y
        1+
        push
        lexical  0,0          ;x
        1-
        push
        fetch    add
        invoke
```

whereas the body of a non-tail-recursive procedure such as

```
(define add
  (lambda (x y)
    (if (0? x)
        y
        (1+ (add (1- x) y)))))
```

compiles as something like

```
        lexical  0,0          ;x
        0?
        branchf  L1           ;branch if nil
        lexical  0,1          ;y
        restore
L1:     save     L2           ;save registers, L2
        lexical  0,1          ;y
        push
        lexical  0,0          ;x
        1-
        push
        fetch    add
        invoke
L2:     1+
        restore
```

Loops in Scheme can be expressed only through tail recursion. The Scheme 311 compiler does not detect loops in order to compile them as backward branches, since loop detection requires an analysis of side effects. For example, if **add** is global then the tail recursive call in the **add** procedure cannot safely be compiled as a backward branch. Hence it is very important that procedure invocation be fast.

The compiler algorithm appears to duplicate the code that follows an if expression. This is because the instruction algebra is treelike. In practice, however, code can be represented as dags rather than as trees, so the subtrees can share one data structure. The Scheme 311 compiler does not generate code with cycles, but similar remarks would apply if it did [30].

Optimisations.

Problems arise when the compiler algorithm to be proved correct includes optimizations that upset the pattern of storage allocation or depend on invariants difficult to express using standard semantics. This section illustrates these problems by considering peephole optimizations at both the source [2] and target levels.

Some source level optimizations can be proved correct simply by proving equalities such as

$$\mathcal{E}[\![\texttt{(if (null? } E_0\texttt{) } E_1\ E_2\texttt{)}]\!]\rho^{init} = \mathcal{E}[\![\texttt{(if } E_0\ E_2\ E_1\texttt{)}]\!]\rho^{init}$$

and

$$\mathcal{E}[\![\texttt{((lambda () E))}]\!] = \mathcal{E}[\![\texttt{E}]\!]$$

On the other hand it is impossible to show that

$$\mathcal{E}[\![\texttt{((lambda (x) x) 3)}]\!] = \mathcal{E}[\![\texttt{3}]\!]$$

because `((lambda (x) x) 3)` allocates storage for the variable `x` whereas the second form doesn't, and this difference can affect the final answer. For example, the initial continuation might print statistics on storage allocation. Perhaps we don't care about such continuations, but the semantics has no way of knowing that. Furthermore it is not immediately obvious how to express our insouciance in terms the semantics can understand.

It is possible to deal with this problem by descending to a lower level of abstraction known as a store semantics and to perform a messy congruence proof [11], but a better way is needed if proofs of compiler correctness are to become routine.

Figure 5. Compiling functions

t: Exp $\to U_C \to$ P $\to$ P — expression compiler, tidy
c: Exp $\to U_C \to$ P $\to$ P — expression compiler
c^*: Exp* $\to U_C \to$ P $\to$ P — argument list compiler

$$t[\![\mathrm{K}]\!] = c[\![\mathrm{K}]\!]$$

$$t[\![\mathrm{I}]\!] = c[\![\mathrm{I}]\!]$$

$$t[\![\texttt{(set! I E)}]\!] = c[\![\texttt{(set! I E)}]\!]$$

$$t[\![\texttt{(lambda (I*) E)}]\!] = c[\![\texttt{(lambda (I*) E)}]\!]$$

$$t[\![\texttt{(if } E_0\ E_1\ E_2\texttt{)}]\!] = \lambda\rho_C\pi\ .\ t[\![E_0]\!]\rho_C(if(t[\![E_1]\!]\rho_C\pi)(t[\![E_2]\!]\rho_C\pi))$$

$$t[\![\texttt{(}E_0\ E^*\texttt{)}]\!] = \lambda\rho_C\ .\ primop[\![E_0]\!](\#E^*)\rho_C \to c[\![\texttt{(}E_0\ E^*\texttt{)}]\!]\rho_C,\ save(c[\![\texttt{(}E_0\ E^*\texttt{)}]\!]\rho_C restore)$$

$$t[\![\texttt{(begin } E_0\ E_1\texttt{)}]\!] = \lambda\rho_C\pi\ .\ t[\![E_0]\!]\rho_C(t[\![E_1]\!]\rho_C\pi)$$

$$c[\![\mathrm{K}]\!] = \lambda\rho_C\ .\ const(K[\![\mathrm{K}]\!])$$

$$c[\![\mathrm{I}]\!] = \lambda\rho_C\ .\ \rho_C\mathrm{I} \in \mathrm{F} \to const(\rho_C\mathrm{I} \mid \mathrm{F}\ \mathrm{in}\ \mathrm{E}),\ \rho_C\mathrm{I} \in \mathrm{Ide} \to fetch(\rho_C\mathrm{I} \mid \mathrm{Ide}), lexical(\rho_C\mathrm{I} \mid \mathrm{N} \times \mathrm{N})$$

$$c[\![\texttt{(set! I E)}]\!] = \lambda\rho_C\pi\ .\ t[\![\mathrm{E}]\!]\rho_C(\rho_C\mathrm{I} \in \mathrm{F} \to illegal,\ \rho_C\mathrm{I} \in \mathrm{Ide} \to set(\rho_C\mathrm{I} \mid \mathrm{Ide})\pi,\ setlex(\rho_C\mathrm{I} \mid \mathrm{N} \times \mathrm{N})\pi)$$

$$c[\![\texttt{(lambda (I*) E)}]\!] = \lambda\rho_C\ .\ lambda(\#\mathrm{I}^*)(c[\![\mathrm{E}]\!](extends_C\rho_C\mathrm{I}^*)restore)$$

$$c[\![\texttt{(if } E_0\ E_1\ E_2\texttt{)}]\!] = \lambda\rho_C\pi\ .\ t[\![E_0]\!]\rho_C(if(c[\![E_1]\!]\rho_C\pi)(c[\![E_2]\!]\rho_C\pi))$$

$$c[\![\texttt{(}E_0\ E^*\texttt{)}]\!] = \lambda\rho_C\pi\ .\ primop[\![E_0]\!](\#E^*)\rho_C \to p[\![\texttt{(}E_0\ E^*\texttt{)}]\!]\rho_C\pi,\ c^*[\![E^*]\!]\rho_C(t[\![E_0]\!]\rho_C invoke)$$

$$c[\![\texttt{(begin } E_0\ E_1\texttt{)}]\!] = \lambda\rho_C\pi\ .\ t[\![E_0]\!]\rho_C(c[\![E_1]\!]\rho_C\pi)$$

$$c^*[\![\ ]\!] = \lambda\rho_C\pi\ .\ \pi$$

$$c^*[\![E^*\ E]\!] = \lambda\rho_C\pi\ .\ t[\![E]\!]\rho_C(push(c^*[\![E^*]\!]\rho_C\pi))$$

$$primop = \lambda E\nu\rho_C\ .\ E \in \mathrm{Ide} \to \nu \neq 0 \to \rho_C(E \mid \mathrm{Ide}) \in \mathrm{F}, false, false$$

$$p[\![\texttt{(I } E\ E^*\texttt{)}]\!] = \lambda\rho_C\pi\ .\ c^*[\![E^*]\!]\rho_C(t[\![E]\!]\rho_C(op(\#E^*)(\rho_C\mathrm{I} \mid \mathrm{F})\pi))$$

Exactly the same problem appears at the target level. It is trivial to prove the correctness of such target level peephole optimizations as

$$fetch\,1\,(if\,(fetch\,1\,\pi_1)\,\pi_2) = fetch\,1\,(if\,\pi_1\,\pi_2)$$

and

$$fetch\,1\,(push\,(fetch\,1\,\pi)) = fetch\,1\,(push\,\pi)$$

Most optimizations of the sort described in [10] are of this nature. It so happens that these optimizations are relatively unimportant when compared with optimizations that disturb the pattern of storage allocation, as for example when a variable is kept in a register rather than on a heap. Such target level optimizations present the same obstacles to correctness proof as source level optimizations that alter storage allocation.

More global optimizations depend on more global analyses, such as a side effects analysis. It is not clear whether such analysis can be performed within standard semantics. Once again the standard solution is to descend to a store semantics.

With stack-based languages a deletion strategy can be moved into the semantics of the language. For example the semantics might specify that on return from a procedure the storage allocated by that procedure is reclaimed. Such a tactic is only a partial solution to the problems that have been outlined in this section, and absent a proof it begs the question of whether the resulting semantics is equivalent to the original semantics.

Until these problems are solved, it is reasonable for compiler designers to rely on conjectured congruences. Designers concerned to verify their compilers will prove the conjectures, using store semantics if necessary, but even if left unproved a careful statement of the conjectures will serve to warn of thin ice.

Related Work.

The problem of proving a compiler algorithm correct relative to a high level specification such as a standard semantics is very different from the problem of generating a compiler from the specification. For example, [18] describes an automatic system that can often transform a denotational semantics into a reasonably efficient code generator, but the correctness of the code generators so obtained is a conjecture rather than a theorem. As is pointed out in [16], present-day automatic systems that can guarantee correctness relative to a standard semantics also generate inefficient code. Work in progress threatens to alter that assessment, however [4].

The correctness proof reported here falls into a tradition of simple, rigorous compiler correctness proofs spanning more than a decade [3, 12, 13, 27, 28]. This paper is the first in that line to prove the correctness of a usable compiler for a real programming language. (It helps of course that Scheme is simpler even than many toy languages and that the overhead from decoding instructions in software masks inefficiencies.) The importance of compiler correctness proofs for real programming languages is that they try the ability of the proof techniques to adapt to unanticipated difficulties. For example, the byte code implementation based on the Scheme 311 algorithm tries to keep variables in registers rather than on the heap, but that optimization could not be included in this paper because there is no known simple proof of its correctness.

This paper is based directly on [29]. One difference is that in this paper the target machine instructions perform their own sequencing, while [29] uses special sequencing combinators to paste actions together. This difference is not in itself terribly important, since the target machines in [29] are reasonable only if pairs consisting of a sequencing combinator and an action are viewed as single instructions. The real advantage of the special sequencing combinators is that they allow separate inductions to be used to prove certain associativities satisfied by the combinators and to prove the correctness of algorithms for linearizing the target code, "optimizing" tail recursion, and distributing compile-time environment information. In this paper all those inductions are performed at once, mainly because it was easier to work with the compiler algorithm as it existed than to tear it apart into distinct phases.

Difficulties in reasoning about storage allocation within denotational semantics were noted recently in [8], which puts forth a novel treatment of storage allocation. The notion of storage allocation used in the semantics of Scheme and in the compiler correctness proof is the traditional notion in which *new* is continuous. (See sections 3.3.2 and 4.2.6 of [11].)

Scheme was first described in [25], which was revised by [24]. An interesting optimizing compiler for Scheme using ideas from continuation semantics appears in [21]. No production quality implementations of full Scheme now exist although T, developed at Yale, comes close [19]. Common Lisp [23] was influenced by Scheme and contains many of its good features, but ultimately Scheme's simplicity was sacrificed to the goal of compatibility with existing Lisps. Most implementations of Scheme have been based on the idea of compiling to an SECD machine code. Microelectronic chips have been developed to interpret one such code [26].

Conclusion.

The Scheme 311 compiler algorithm has been stated and proved correct relative to a standard semantics for Scheme. This algorithm has been used in several implementations including Scheme 311 Versions 2 through 4 [5], Scheme 84 [6], and a byte code implementation for the Motorola 68000. The main benefits of the proof are that it points out how the source language was compromised, makes explicit the global invariants on which the algorithm depends, and provides the semantic framework needed to explore new optimizations and architectures.

The design of the target machine may have been the hardest part of the compiler construction and proof. The target machine itself is just a special purpose algebra. The reason it is difficult to extract such an algebra from the source language's semantics is that the desire for efficiency leads the target machine's architecture away from the obvious algebra. The process by which the target machine is designed should be seen as a descent to lower levels of abstraction in which higher level data types are implemented in terms of more machine-like concepts. It is important to describe the target machine at as high a level as possible, however, both to simplify the compiler correctness proof and to preserve design flexibility for lower levels of implementation. Continuation semantics provides a nice meeting ground between the source language and the target machine.

The idea of factoring environments through a domain of compile-time environments works well for Scheme and should be tried with statically typed languages.

The main problem with the semantic framework used in this paper is that it cannot deal with many important optimizations, including the vitally important optimization that uses a deletion strategy for environments in stack-based languages. Store semantics is required, and the proofs tend to be tedious.

Acknowledgements.

Preparation of this report was supported by NSF grant MCS 83-03325.

I wish to thank Steve Johnson for reading and commenting a draft of this paper.

References.

[1] Urs Amman, P Nori, K Jensen, H Nageli, "The Pascal (P) compiler implementation notes", Institut für Informatik, Eidgenossische Technische Hochschule, Zurich, 1975.

[2] Rodney A Brooks, Richard P Gabriel, and Guy L Steele Jr, "An optimizing compiler for lexically scoped Lisp", in the *Proceedings of the SIGPLAN '82 Symposium on Compiler Construction*, June 1982, pages 261–275.

[3] R M Burstall and P J Landin, "Programs and their proofs: an algebraic approach", *Machine Intelligence* **4**, 1969, pages 17–44.

[4] Henning Christiansen and Neil Jones, "Control flow treatment in a simple semantics-directed compiler generator", *Formal Description of Programming Concepts II* (Preliminary Proceedings), IFIP TC-2 Working Conference, Garmisch-Partenkirchen, June 1982, pages 38–62.

[5] Carol Fessenden, William Clinger, Daniel P Friedman, and Christopher Haynes, "Scheme 311 Version 4 Reference Manual", Indiana University Computer Science Technical Report 137, February 1983.

[6] Daniel P Friedman, Christopher T Haynes, Eugene Kohlbecker, and Mitchell Wand, "Scheme 84 Version 0 Programming Language Reference Manual", Indiana University Computer Science Technical Report 153, February 1984.

[7] A Goldberg and D Robson, *Smalltalk-80: The Language and its Implementation*, Addison-Wesley, 1983.

[8] Joseph Y Halpern, Albert R Meyer, B A Trakhtenbrot, "The semantics of local storage, or what makes the free-list free? (preliminary report)", *ACM Symposium on Principles of Programming Languages*, 1984, pages 245–257.

[9] Scott Kamins, "Macintosh BASIC", *Byte* **9**, 4, April 1984, pages 318–330.

[10] Larry M Masinter and L Peter Deutsch, "Local optimization in a compiler for stack-based Lisp machines", *Conference Record of the 1980 Lisp Conference*, August 1980, pages 223–230.

[11] Robert Milne and Christopher Strachey, *A Theory of Programming Language Semantics*, Halstead Press, 1976.

[12] R Milner and R Weyrauch, "Proving compiler correctness in a mechanized logic", *Machine Intelligence* **7**, 1972, pages 51–72.

[13] F L Morris, "Advice on structuring compilers and proving them correct", *ACM Symposium on Programming Languages*, 1973, pages 144–152.

[14] Peter Mosses, "Abstract semantic algebras!", *Formal Description of Programming Concepts II* (Preliminary Proceedings), IFIP TC-2 Working Conference, Garmisch-Partenkirchen, June 1982, pages 63–88.

[15] Steven S Muchnik and Uwe Pleban, "A semantic comparison of Lisp and Scheme", *Conference Record of the 1980 Lisp Conference*, 1980, pages 56–64.

[16] James Bodwin, Laurette Bradley, Kohji Kanda, Diane Litle, and Uwe Pleban, "Experience with an experimental compiler generator based on denotational semantics", *Proceedings of the SIGPLAN '82 Symposium on Compiler Construction*, June 1982, pages 216–229.

[17] Wolfgang Polak, *Compiler Specification and Verification*, Springer-Verlag Lecture Notes in Computer Science 124, 1981.

[18] Martin R Raskovsky, "Denotational semantics as a specification of code generators", *Proceedings of the SIGPLAN '82 Symposium on Compiler Construction*, June 1982, pages 230–244.

[19] Jonathan A Rees and Norman I Adams IV, "T: a dialect of Lisp; or Lambda: the ultimate software tool", *ACM Symposium on Lisp and Functional Programming*, 1982, pages 114–122.

[20] Brian Cantwell Smith, "Reflection and semantics in Lisp", *Conference Record of the Eleventh Annual ACM Symposium on Principles of Programming Languages*, January 1984, pages 23–35.

[21] Guy Lewis Steele Jr, "Rabbit: a compiler for Scheme [...]", MIT Artificial Intelligence Laboratory Technical Report 472, May 1978.

[22] Guy Lewis Steele Jr, "Debunking the "Expensive Procedure Call" myth; or, procedure call implementations considered harmful; or LAMBDA: the ultimate GOTO", *ACM-77*, 1977, pages 153–162.

[23] Guy L Steele, "An overview of Common Lisp", *ACM Symposium on Lisp and Functional Programming*, 1982, pages 98–107.

[24] Guy Lewis Steele Jr and Gerald Jay Sussman, "The revised report on Scheme, a dialect of Lisp", MIT Artificial Intelligence Laboratory Memo 452, January 1978.

[25] Gerald Jay Sussman and Guy Lewis Steele Jr, "Scheme: an interpreter for extended lambda calculus", MIT Artificial Intelligence Laboratory Memo 349, December 1975.

[26] Gerald J Sussman, Jack Holloway, Guy L Steele, and Alan Bell, "Scheme-79—Lisp on a chip", *IEEE Computer* **14**, 7, July 1981, pages 10–21.

[27] James W Thatcher, Eric G Wagner, and Jesse B Wright, "More on advice on structuring compilers and proving them correct", in H A Maurer [ed], *Automata, Languages, and Programming, 6th Colloquium, Graz, July 1979*, Springer-Verlag Lecture Notes in Computer Science 71, pages 596–615.

[28] Mitchell Wand, "Semantics-directed machine architecture", *ACM Symposium on Principles of Programming Languages*, 1982, pages 234–241.

[29] Mitchell Wand, "Deriving target code as a representation of continuation semantics", *ACM Transactions on Programming Languages and Systems* **4**, 3, July 1982, pages 496–517.

[30] Mitchell Wand, "Loops in combinator-based compilers", *ACM Symposium on Principles of Programming Languages*, 1983, pages 190–196.